

RoMeRL: BALANCING FEEDBACK COVERAGE AND THE MEMORY-REWARD TRAP IN SELF-EVOLVING AGENT MEMORY VIA REDUCED-ORDER UTILITY STATES

Yi Yang^{1,*}, Zhennan Chen^{1,*†}, Yihong Zhuang², Tiehan Fan¹,
Yinan Chen³, Jian Li¹, Jian Yang¹, Ying Tai^{1,‡}

¹Nanjing University ²Xiamen University ³Zhejiang University

ABSTRACT

Learning-based memory systems for self-evolving LLM agents face two tightly coupled challenges. First, trajectory-indexed utilities grow with the interaction history, thereby dispersing limited feedback over an ever-expanding state space. Second, because trajectory-level rewards are jointly assigned to co-retrieved memories, irrelevant experiences may receive misleading utility updates and consequently enter the *memory-reward trap*. To address these challenges, we introduce Reduced-Order Memory Reinforcement Learning (RoMeRL), which represents the growing trajectory-indexed utility space using a fixed-dimensional per-task memory state factorized by outcome polarity and memory dynamics. RoMeRL incorporates new experiences through a fixed set of semantic coordinates whose contents are updated or replaced over time, thereby concentrating feedback over a bounded utility support. Theoretically, we show that this reduced-order parameterization increases the average feedback received by each utility coordinate and characterize the steady-state occupancy of erroneous coordinates under a generic coordinate-transition model. Empirically, across ALFWorld and LifelongAgentBench, RoMeRL improves task performance, reduces the Cold-Q ratio by 80.0%, increases feedback density by approximately 6.0 $\times$, reduces the maintained memory size by 84.4%, and cuts LLM calls by 21.1%. These results show that reduced-order utility states support efficient self-evolving agent memory while limiting persistent reward contamination. Code is available at: <https://github.com/YOUNG-fnxm/RoMeRL>

1 INTRODUCTION

Large language model (LLM) agents are inherently stateless, which limits their ability to accumulate and reuse experience across interactions (Sumers et al., 2023; Tao et al., 2024). Agent memory addresses this limitation by storing and retrieving past experience; recent surveys organize this design space by memory substrate, cognitive role, operational lifecycle, and externalized agent infrastructure (Zhang et al., 2025b; Huang et al., 2026; Zhou et al., 2026). Early methods maintain episodic memories or reusable skill libraries and retrieve trajectories by semantic similarity (Zhong et al., 2024; Cai et al., 2025; Wang et al., 2023). Later systems introduce explicit memory lifecycles, using handcrafted workflows, reflection, summarization, and rule-based selection to organize and refine agent experience (Packer et al., 2023; Fu et al., 2024; Ouyang et al., 2025; Zhao et al., 2024; Wang et al., 2024). Recent learning-based approaches instead optimize memory generation, retrieval, and utility from downstream task outcomes, allowing the memory system to evolve through interaction without updating the underlying LLM (Yan et al., 2026; Zhou et al., 2025; Zhang et al., 2025a; 2026). The focus has consequently shifted from preserving past experience to deciding which experiences should remain active and affect future behavior.

Existing end-to-end memory optimization methods commonly assign a separate utility to every stored trajectory and update it from downstream task outcomes. As experience accumulates, this trajectory-indexed formulation continually expands the dimensionality of the learnable memory state

*Equal contribution.

†Project Leader.

‡Corresponding Author.

while feedback remains limited, resulting in widespread utility cold start, concentrated updates, and low feedback density. A natural remedy is to increase exploration so that under-visited memories receive more feedback. However, our experiments with Upper Confidence Bound (UCB) show that, although stronger exploration improves memory coverage and alleviates cold start, it degrades task performance. Because trajectory-level rewards are jointly assigned to co-retrieved memories, broader exploration also exposes more weakly relevant memories to successful contexts, allowing them to receive positive updates without corresponding contributions. We refer to this exploration–contamination dilemma as the *Memory-Reward Trap (MRT)*, illustrated in Figure 1.

This conflict raises a more fundamental question:

How can an agent improve memory-feedback coverage without expanding the utility support exposed to erroneous reward propagation? We

trace this dilemma to trajectory-indexed utility learning: every newly stored experience introduces an additional utility variable, while broader exploration exposes more such variables to potentially misattributed rewards. To address this problem, we propose Reduced-Order Memory Reinforcement Learning (RoMeRL). Instead of exploring a continually expanding utility space more aggressively, RoMeRL changes the state on which memory reinforcement learning operates. It replaces trajectory-indexed utilities with a fixed-dimensional per-task state factorized by outcome polarity and memory dynamics. Outcome polarity separates positive from negative evidence; memory dynamics separates consolidated historical evidence from adaptive state-transition evidence. For each task, their Cartesian product defines a small set of persistent semantic coordinates whose contents are updated or replaced as new experiences arrive. The resulting state concentrates feedback while limiting the dimensionality and persistence of support exposed to the memory-reward trap.

We evaluate RoMeRL on ALFWorld (Shridhar et al., 2020) and LifelongAgentBench (Zheng et al., 2025), which cover embodied planning, operating-system interaction, and database operation. RoMeRL improves task performance, reduces the Cold-Q ratio by 80.0%, and increases feedback density by approximately $6.0\times$. The reduced-order state also decreases the maintained memory size by 84.4% and LLM calls by 21.1%, without updating the underlying LLM.

Our contributions are as follows:

- We show that trajectory-indexed memory learning dilutes feedback as the utility state grows and expands the persistent support exposed to the memory-reward trap.
- We propose RoMeRL, which replaces each task’s growing memory utility space with a fixed-dimensional state factorized by outcome polarity and memory dynamics.
- We characterize the balance between feedback and the memory-reward trap in reduced-order utility states, and evaluate RoMeRL’s effectiveness and efficiency across agent benchmarks.

2 RELATED WORK

Workflow-Based Agent Memory. These systems build on retrieval-augmented inference, where external evidence is selected by learned relevance models and supplied as model context (Karpukhin et al., 2020; Lewis et al., 2020). Early agent-memory systems externalize interaction history through predefined storage, reflection, and retrieval workflows. Generative Agents and MemoryBank maintain episodic streams with importance-based retrieval, reflection, and forgetting mechanisms (Park et al., 2023; Zhong et al., 2024). Reflexion and ExpeL convert task feedback into verbal reflections or transferable insights (Shinn et al., 2023; Zhao et al., 2024), while Voyager and AutoGuide distill interactions into reusable skills or context-dependent guidelines (Wang et al., 2023; Fu et al., 2024).

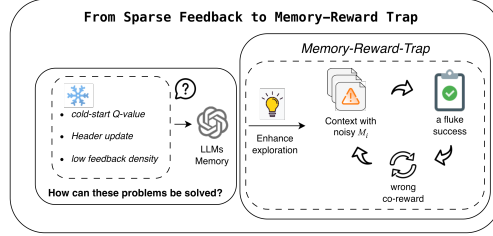


Figure 1: **Memory-reward trap:** Stronger exploration retrieves more low-access-count memories, including weakly relevant, noisy, outdated, or failed experiences. When such a memory appears in a successful episode, the shared trajectory reward can promote it despite no causal contribution. Exploration therefore increases feedback coverage but also raises the risk of reward misattribution.

MemGPT and HiAgent provide virtual or hierarchical memory management (Packer et al., 2023; Hu et al., 2025); A-MEM links structured notes into an evolving knowledge network (Xu et al., 2026). These methods avoid model tuning, but their memory decisions are largely governed by handcrafted lifecycles, prompting rules, and semantic similarity rather than downstream task outcomes.

Learning-Based Agent Memory. Recent work treats agent memory as a learnable component rather than a static retrieval store. ReasoningBank and MemP distill trajectories into evolving reasoning or procedural memories (Ouyang et al., 2025; Fang et al., 2026). Memory-R1, Agentic Memory, and AtomMem use reinforcement learning to train memory construction, retrieval, update, and deletion policies (Yan et al., 2026; Yu et al., 2026; Yao et al., 2026). Fine-Mem assigns fine-grained rewards to individual memory operations (Ma et al., 2026), whereas MemEvolve adapts the memory architecture across tasks (Zhang et al., 2025a). MemRL, the closest setting to ours, learns episodic-memory utilities through non-parametric runtime reinforcement learning (Zhang et al., 2026). These methods generally optimize a growing collection of memory entries; RoMeRL instead represents each task’s utility-bearing state with a fixed number of semantic coordinates.

3 PRELIMINARIES

Reinforcement Learning in Agentic Memory. Outcome-driven agent memory methods associate each stored trajectory m_i with a learnable utility $Q_{i,t}$ and use the utility together with semantic relevance for memory retrieval Salama et al. (2025); Zhang et al. (2025c; 2026). Let $\mathcal{M}_t = \{(m_i, Q_{i,t})\}_{i=1}^{N_t}$ denote the memory bank at interaction step t , and let $\mathcal{S}_t \subseteq \mathcal{M}_t$ be the memories retrieved for the current query. After the agent completes the trajectory and receives task-level reward r_t , the utility of each retrieved memory is updated as

$$Q_{i,t+1} = Q_{i,t} + \alpha \mathbb{I}[m_i \in \mathcal{S}_t](r_t - Q_{i,t}), \quad (1)$$

where α is the learning rate. This formulation improves the agent by updating external memory utilities rather than the parameters of the underlying LLM. However, because every newly stored trajectory introduces an additional utility variable, the learnable memory state $\mathbf{Q}_t = (Q_{1,t}, \dots, Q_{N_t,t}) \in \mathbb{R}^{N_t}$ grows continuously with the interaction history.

4 REDUCED-ORDER MEMORY RL

In this section, we formulate end-to-end memory reinforcement learning as utility learning over a growing trajectory-indexed state. Bundle-level rewards estimate observational rather than marginal utility, giving rise to the *memory-reward trap*, while reliable full-state estimation requires feedback that grows with the number of stored trajectories. Together, these properties create an exploration dilemma: broader coverage exposes more utility variables to misattributed rewards. RoMeRL addresses both problems by replacing each task’s growing index set with a fixed-dimensional state factorized by outcome polarity and memory dynamics. We first analyze feedback concentration and erroneous-coordinate occupancy for a general active dimension, and then present the practical implementation. The overall architecture of RoMeRL is shown in Figure 2.

4.1 TRAJECTORY-INDEXED FEEDBACK AND THE MEMORY-REWARD TRAP

Under the trajectory-indexed parameterization, each stored trajectory m_i corresponds to an independently learned utility Q_i , whereas the task-level reward is jointly determined by the complete retrieved context $\mathcal{S}_t$. To distinguish the utility estimated from bundle-level outcomes from the individual contribution of a memory, we introduce the following definition.

Definition 1 (Trajectory-Indexed Memory Utility). *Let $n_i = \sum_t \mathbb{I}[m_i \in \mathcal{S}_t]$, $\hat{Q}_i = \frac{1}{n_i} \sum_{t:m_i \in \mathcal{S}_t} R_t$, $n_i > 0$, denote the retrieval count and empirical utility of memory m_i . Its observational utility is $\mu_i = \mathbb{E}[R_t \mid m_i \in \mathcal{S}_t]$. Let the interventional outcomes with and without m_i be $v_i^1 = \mathbb{E}[R_t \mid \text{do}(m_i \in \mathcal{S}_t)]$, $v_i^0 = \mathbb{E}[R_t \mid \text{do}(m_i \notin \mathcal{S}_t)]$, where the task distribution and remaining retrieval context are fixed. The marginal utility, observational attribution bias, and total credit gap are respectively*

$$\begin{aligned} \theta_i &= v_i^1 - v_i^0, a_i = \mu_i - v_i^1, \\ \mu_i - \theta_i &= v_i^0 + a_i. \end{aligned} \quad (2)$$

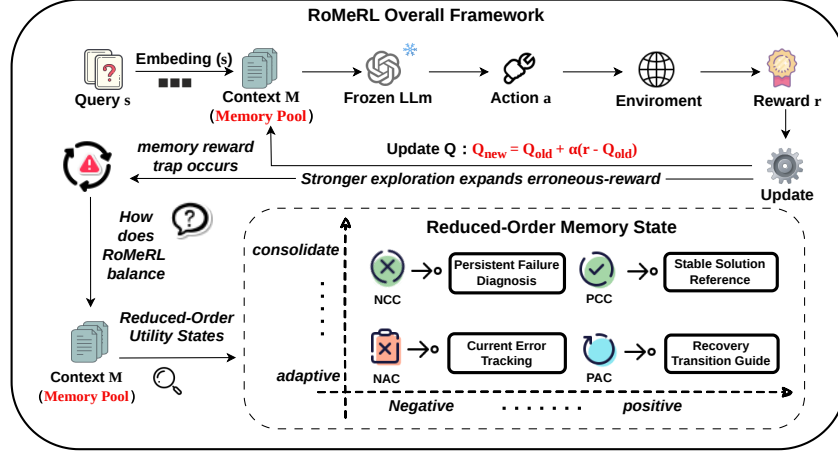


Figure 2: Overview of RoMeRL. **Top:** the agent retrieves memories to construct the LLM context and updates memory utilities using task-level rewards. Although stronger exploration improves feedback coverage, it also expands the utility support exposed to erroneous reward propagation, increasing the risk of the memory-reward trap. **Bottom:** RoMeRL maps each task’s growing trajectory-indexed utility space into a fixed-dimensional state factorized by outcome polarity and memory dynamics. Its four semantic coordinates PCC, PAC, NCC, and NAC are updated through online retention, promotion, and replacement, thereby concentrating feedback over a bounded active support.

Here, v_i^0 is the task-level baseline, while a_i captures attribution induced by retrieval selection, co-retrieved memories, context interactions, and task-level confounders.

Because only bundle-level rewards are observed, $\hat{Q}_i$ estimates the raw-return target μ_i , rather than the marginal contribution θ_i . Their gap consists of the task-level baseline v_i^0 and the observational attribution bias a_i .

We next characterize how the credit gap and the number of observations jointly determine the accuracy of trajectory-indexed utility estimation.

Theorem 1 (Gap–Variance Decomposition of Memory Utility). *Suppose that the rewards observed when m_i is retrieved are conditionally independent samples with mean μ_i and variance σ_i^2 : $\mathbb{E}[R_t | m_i \in S_t] = \mu_i$, $\text{Var}(R_t | m_i \in S_t) = \sigma_i^2$. Then*

$$\begin{aligned} \mathbb{E}[(\hat{Q}_i - v_i^0)^2] &= a_i^2 + \frac{\sigma_i^2}{n_i}, \\ \mathbb{E}[(\hat{Q}_i - \theta_i)^2] &= (v_i^0 + a_i)^2 + \frac{\sigma_i^2}{n_i}. \end{aligned} \quad (3)$$

Proof. See Appendix A.

Theorem 1 separates statistical uncertainty from two distinct sources of credit mismatch. Retrieving m_i more often increases n_i and reduces the variance term, but it does not remove the task-level baseline v_i^0 or the observational attribution bias a_i . Additional feedback therefore makes Q_i estimate the raw-return target μ_i more precisely; it does not turn the raw-return estimate into the memory’s marginal contribution θ_i .

The most harmful case is the **Memory-Reward Trap (MRT)**: a memory with non-positive marginal contribution exhibits a positive observational signal relative to the matched no-memory baseline.

Definition 2 (Memory-Reward Trap). *A memory m_i falls into a memory-reward trap if its interventional marginal utility is non-positive while its observational raw return exceeds the interventional baseline without that memory:*

$$\theta_i \leq 0, \quad \mu_i - v_i^0 = \theta_i + a_i > 0. \quad (4)$$

Thus, a memory with no positive marginal contribution may still have an observational utility above the matched baseline due to retrieval selection, co-retrieved memories, or context interactions. The Q -value estimates μ_i , while $\mu_i - v_i^0$ represents its observational excess over the baseline.

For the exponential update $Q_{i,t+1} = (1 - \alpha)Q_{i,t} + \alpha R_t$, the expected change after retrieving m_i is $\mathbb{E}[Q_{i,t+1} - Q_{i,t} \mid m_i \in \mathcal{S}_t] = \alpha(\mu_i - Q_{i,t}) = \alpha(v_i^0 - Q_{i,t}) + \alpha(\theta_i + a_i)$. The two terms represent baseline drift and the observational signal above that baseline, respectively. Under the MRT, $\theta_i \leq 0 < \theta_i + a_i$, so repeated Q -based retrieval or retention can reinforce a memory with non-positive marginal utility.

Let $\mathcal{B}_t = \{m_i \in \mathcal{M}_t : \theta_i \leq 0, \mu_i - v_i^0 > 0\}$, $\rho_t = \Pr(m_i \in \mathcal{B}_t)$. Then $\mathbb{E}[|\mathcal{B}_t|] = N_t \rho_t$, so the expected support exposed to erroneous reward propagation grows with the memory-state dimension N_t .

5 REDUCED-ORDER MEMORY STATE

The preceding analysis separates raw-return estimation from marginal credit assignment. We now consider a complementary question: how much feedback is required to estimate raw-return utilities over a growing trajectory-indexed state? As each stored trajectory introduces an additional utility variable, the dimensionality of this state grows continuously with the interaction history.

Assumption 1 (Full-Pool Raw-Return Estimation Setting). *To isolate the effect of utility-state dimensionality, assume that each stored memory has a stationary raw-return target μ_i . The goal is to estimate every stored target within error ϵ with probability at least $1 - \delta$:*

$$\Pr\left(\forall i \in [N_t], |\hat{Q}_i - \mu_i| \leq \epsilon\right) \geq 1 - \delta. \quad (5)$$

Let $F_T = \sum_{i=1}^{N_t} n_i$ be the total number of memory-level updates. If each trajectory updates at most k memories, then $F_T \leq kT$.

This setting characterizes the feedback required to estimate a growing collection of raw-return utilities. Their relation to the corresponding marginal contributions is captured by the credit gap $v_i^0 + a_i$.

Theorem 2 (Sufficient Feedback Budget for Trajectory-Indexed Utilities). *Suppose that rewards are bounded in $[0, 1]$ and that each $\hat{Q}_i$ is estimated from n_i independent samples with mean μ_i . By Hoeffding's inequality and a union bound, Equation (5) holds if $n_i \geq \frac{1}{2\epsilon^2} \log \frac{2N_t}{\delta}$, $\forall i \in [N_t]$. Consequently, sufficient feedback and trajectory budgets scale as*

$$F_T = O\left(\frac{N_t}{\epsilon^2} \log \frac{N_t}{\delta}\right), \quad T = O\left(\frac{N_t}{k\epsilon^2} \log \frac{N_t}{\delta}\right). \quad (6)$$

Proof. See Appendix A.

Relation to marginal contribution. Theorem 2 concerns estimation of the raw-return targets μ_i . Relative to the marginal contributions $|\hat{Q}_i - \theta_i| \leq |\hat{Q}_i - \mu_i| + |v_i^0 + a_i|$. Let $G_t = \max_{i \in [N_t]} |v_i^0 + a_i|$ denote the maximum total credit gap. When $G_t < \epsilon$, the same concentration argument gives a sufficient feedback budget after replacing ϵ with $\epsilon - G_t$. Importantly, G_t contains both the task-level baseline v_i^0 and the observational attribution bias a_i ; additional feedback reduces estimation uncertainty around μ_i but does not remove either component.

Theorem 2 isolates the dimension-dependent cost of full-pool estimation, while the credit-gap relation shows that broader exploration cannot remove the task baseline or observational attribution component.

We therefore replace the N_t -dimensional trajectory-indexed utility space with a fixed 2×2 state factorized by outcome polarity and memory dynamics:

$$\mathcal{O} = \{+, -\}, \quad \mathcal{D} = \{C, A\}, \quad (7)$$

Here, $+$ and $-$ denote successful and failed evidence, while C and A denote consolidated and adaptive dynamics. Consolidated coordinates retain globally selected evidence; adaptive coordinates track the current state or meaningful transitions. Their Cartesian product gives

$$\mathcal{I}^{\text{fact}} = \mathcal{O} \times \mathcal{D}, \quad |\mathcal{I}^{\text{fact}}| = 4. \quad (8)$$

For task g , the variable-length history $D_{g,t}$ is mapped to

$$\mathbf{Z}_{g,t} = \Phi(D_{g,t}) = \left[z_{g,t}^{o,d} \right]_{(o,d) \in \mathcal{O} \times \mathcal{D}} = \begin{bmatrix} z_{g,t}^{+,C} & z_{g,t}^{+,A} \\ z_{g,t}^{-,C} & z_{g,t}^{-,A} \end{bmatrix}. \quad (9)$$

Coordinate contents may change, but their semantic identities remain fixed. Each new trajectory therefore updates or replaces a coordinate rather than adding a persistent utility variable:

$$\mathbf{Z}_{g,t+1} = \mathcal{U}(\mathbf{Z}_{g,t}, m_{t+1}, R_{t+1}), \quad \dim(\mathbf{Z}_{g,t}) = 4. \quad (10)$$

The 2×2 state is the smallest complete product of these binary distinctions. The following results hold for a general active dimension d , while $d = 4$ is the design induced by this factorization. We next analyze its feedback allocation.

Theorem 3 (Feedback Concentration under State Reduction). *Consider a utility-bearing memory state with dimension d . Under interaction budget T , per-trajectory update limit k , and approximately balanced feedback allocation, each utility coordinate receives $\bar{n}_d \approx \frac{kT}{d}$ feedback signals on average. Therefore, trajectory-indexed learning with dimension N_t and factorized learning with dimension 4 satisfy $\bar{n}_{\text{fact}} = \frac{N_t}{4} \bar{n}_{\text{full}}$. If the reward variance is bounded by σ^2 , the corresponding average utility-estimation variance changes:*

$$\text{from } O\left(\frac{\sigma^2 N_t}{kT}\right) \quad \text{to} \quad O\left(\frac{4\sigma^2}{kT}\right). \quad (11)$$

Proof. See Appendix A.

Theorem 3 shows that state reduction concentrates a fixed feedback budget over fewer utilities, whereas full-pool exploration only redistributes feedback across a growing set.

A smaller active state also limits simultaneous exposure to misleading feedback, while its persistence depends on the contamination and replacement dynamics characterized by the following proposition.

Proposition 1 (Erroneous-Coordinate Occupancy under a Transition Model). *Let $\rho_{\text{full}} = \Pr(\theta_i \leq 0, \mu_i - v_i^0 > 0)$ be the trap probability of a trajectory-indexed utility, so that the expected number of trap-affected utilities in a full pool of size N_t is $N_t \rho_{\text{full}}$. Consider a generic active state of dimension d in which each coordinate follows stationary clean–erroneous transitions. Suppose that the clean-to-erroneous transition probability is at most γ and the erroneous-to-clean transition probability is at least $\lambda > 0$. Then the steady-state erroneous fraction is at most $\gamma/(\gamma + \lambda)$, and the expected number of erroneous active coordinates is at most $d \frac{\gamma}{\gamma + \lambda}$. This occupancy is lower than the expected full-pool exposure whenever*

$$d \frac{\gamma}{\gamma + \lambda} < N_t \rho_{\text{full}}. \quad (12)$$

Proof. See Appendix A.

Theorem 3 characterizes feedback concentration under a bounded active dimension. Proposition 1 separates two factors governing persistent erroneous occupancy: the active dimension d controls the maximum number of exposed coordinates, while γ and λ describe the contamination and correction dynamics of a generic replacement process. For a four-coordinate state, the conditional occupancy bound is obtained by setting $d = 4$.

5.1 PRACTICAL IMPLEMENTATION OF RoMERL

We implement RoMERL as an online memory state. For task g , let $\mathcal{D}_{g,t} = \{m_i\}_{i=1}^{n_{g,t}}$ denote the trajectories observed by interaction t , where each m_i has outcome $y_i \in \{0, 1\}$, efficiency ℓ_i , temporal index t_i , and utility Q_i . The factorized state is

$$\mathbf{Z}_{g,t} = \begin{bmatrix} z_{g,t}^{+,C} & z_{g,t}^{+,A} \\ z_{g,t}^{-,C} & z_{g,t}^{-,A} \end{bmatrix}, \quad z_{g,t}^{o,d} = (m_{g,t}^{o,d}, Q_{g,t}^{o,d}). \quad (13)$$

Each non-empty coordinate stores one representative and its utility, defining the active retrieval support

$$\mathcal{A}_{g,t} = \left\{ m_{g,t}^{o,d} : (o, d) \in \mathcal{O} \times \mathcal{D}, m_{g,t}^{o,d} \neq \emptyset \right\}, |\mathcal{A}_{g,t}| \leq 4. \quad (14)$$

Each coordinate starts from Q_{init} . An incoming representative inherits the current utility as a warm start and resets n_i^{post} to zero; subsequent outcomes adapt the inherited value. For query x_t , RoMeRL ranks active memories by weighted similarity and utility:

$$\begin{aligned} \text{score}_t(m_{g,t}^{o,d}) &= (1 - \omega_Q) \cos(e(x_t), e(m_{g,t}^{o,d})) + \omega_Q Q_{g,t}^{o,d}, \\ \mathcal{S}_t &= \text{TopK}_{k_{\text{ret}}}(\mathcal{A}_{g,t}; \text{score}_t). \end{aligned} \quad (15)$$

Here, $e(\cdot)$ is the embedding encoder and $\omega_Q \in [0, 1]$ weights the learned utility. All non-empty coordinates are candidates, and the top k_{ret} memories are retrieved.

Positive Consolidated Coordinate (PCC). The coordinate $z_{g,t}^{+,C}$ preserves a globally consolidated positive reference. Among successful trajectories, it retains the most efficient one:

$$m_{g,t}^{+,C} = \arg \min_{m_i \in \mathcal{D}_{g,t}: y_i=1} \ell_i. \quad (16)$$

When a more efficient successful trajectory is observed, it replaces the current representative and inherits the current PCC utility as a warm start. This preserves the accumulated utility state of the consolidated positive coordinate, while subsequent task-outcome updates adapt it to the new representative.

Positive Adaptive Coordinate (PAC). The coordinate $z_{g,t}^{+,A}$ records a positive transition from failure to success. Let $t_g^{\text{fail}} = \min_{m_i \in \mathcal{D}_{g,t}: y_i=0} t_i$ be the first observed failure time. The adaptive positive coordinate retains the earliest successful trajectory following this failure:

$$m_{g,t}^{+,A} = \arg \min_{\substack{m_i \in \mathcal{D}_{g,t}: y_i=1 \\ t_i > t_g^{\text{fail}}}} t_i. \quad (17)$$

Unlike the PCC, this coordinate is selected by temporal transition rather than global efficiency and therefore captures how the agent first crosses a failure-to-success boundary.

Negative Consolidated Coordinate (NCC). The coordinate $z_{g,t}^{-,C}$ retains failed experience that has accumulated positive downstream utility evidence. A failed trajectory is eligible for consolidation only when its utility exceeds the negative initialization threshold Q_{init}^- . Among eligible failures, the state retains the one with the highest utility:

$$m_{g,t}^{-,C} = \arg \max_{\substack{m_i \in \mathcal{D}_{g,t}: y_i=0 \\ Q_i > Q_{\text{init}}^-}} Q_i. \quad (18)$$

Operationally, a trajectory occupying the negative-adaptive coordinate is promoted to this coordinate when its utility exceeds both Q_{init}^- and the utility of the current consolidated negative representative.

Negative Adaptive Coordinate (NAC). The coordinate $z_{g,t}^{-,A}$ tracks the agent’s current failure state by retaining the most recent failed trajectory:

$$m_{g,t}^{-,A} = \arg \max_{m_i \in \mathcal{D}_{g,t}: y_i=0} t_i. \quad (19)$$

Each newly observed failure replaces the previous coordinate content. This temporal update makes the coordinate responsive to recent errors without allowing every failed trajectory to become a persistent utility variable.

Runtime Utility Update. Given the retrieved set $\mathcal{S}_t \subseteq \mathcal{A}_{g,t}$ and the task-level outcome reward r_t , each retrieved coordinate is updated by

$$Q_{g,t+1}^{o,d} = Q_{g,t}^{o,d} + \alpha \mathbb{1}[m_{g,t}^{o,d} \in \mathcal{S}_t] (r_t - Q_{g,t}^{o,d}). \quad (20)$$

The updated utility is combined with semantic similarity for subsequent retrieval. Suppose that a new representative m_j inherits the initial value $Q_{j,0}$ and has a stationary raw-return target μ_j . After s post-replacement utility updates,

$$\mathbb{E}[Q_{j,s} - \mu_j] = (1 - \alpha)^s (Q_{j,0} - \mu_j). \quad (21)$$

Thus, the inherited value provides a warm start, while subsequent task-outcome feedback progressively adapts the utility to the new representative. RoMeRL retains the standard Q update over a fixed set of semantic coordinates.

Table 1: **Main results over 10 epochs.** We compare RoMeRL with non-learning and learning-based agent-memory baselines. LAB reports **Last-Epoch SR / CSR**, while ALFWorld reports SR for six task types: P&P, examine, clean, heat, cool, and Pick-2. **Overall** is the macro-average of last-epoch SR across the two LAB tasks and six ALFWorld task types. For MemP, MemRL, and RoMeRL, we also report average LLM calls and memory-pool size.

Method	Lifelong Agent Bench		ALFWorld						Overall Avg.	LLM Calls Avg Num.	Memory Avg Num.
	OS Last / CSR	DB Last / CSR	P&P	Examine	Clean	Heat	Cool	Pick-2			
<i>Model</i>	DS-V4-flash	DS-V4-flash	GPT-5.4-mini						—	—	—
No Memory	0.646	0.550	0.883	0.827	0.861	0.850	0.855	0.788	0.783	—	—
Pass@10	— / 0.756	— / 0.906	—	—	—	—	—	—	—	—	—
RAG	0.700 / 0.752	0.556 / 0.844	0.891	0.834	0.868	0.855	0.858	0.796	0.795	—	—
Mem0	0.691 / 0.733	0.575 / 0.841	0.897	0.841	0.873	0.858	0.872	0.805	0.802	—	—
MemP	0.768 / 0.796	0.631 / 0.942	—	—	—	—	—	—	—	570K	45K
MemRL	0.808 / 0.820	0.632 / 0.934	0.908	0.855	0.887	0.865	0.871	0.812	0.830	570K	45K
<i>RoMeRL (ours)</i>	0.824 / 0.838	0.680 / 0.952	0.968	0.957	0.901	0.862	0.880	0.826	0.862	450K	7K

6 EXPERIMENTS

We compare RoMeRL with retrieval-based memory (RAG (Wu et al., 2020), Mem0 Chhikara et al. (2025)), agentic memory (MemP (Fang et al., 2026)), test-time scaling (Pass@ k), and runtime memory reinforcement learning (MemRL (Zhang et al., 2026)), using frozen LLM backbones throughout. The evaluation covers ALFWorld and the OS and DB tasks of LifelongAgentBench. All methods within the benchmark use the same backbone and interaction budget. We report Success Rate (SR) and Cumulative Success Rate (CSR), the proportion of tasks solved at least once across epochs. Appendix B provides additional details.

6.1 MAIN EXPERIMENT

Table 1 shows that RoMeRL achieves the highest overall average success rate of 0.862, outperforming the strongest baseline 0.830, by 3.2 percentage points. It maintains a fixed set of semantic coordinates per task while keeping the LLM backbone frozen.

Feedback Utilization, MRT Robustness, and Efficiency. Figure 3 reveals contrasting feedback dynamics. MemRL’s Cold-Q ratio rises from approximately 29% to 44.9%, whereas RoMeRL reduces it from approximately 28% to 9.0% and increases feedback density from 4.96 to 29.93 (6.0 $\times$). This improvement in feedback utilization is accompanied by an 84.4% reduction in average memory size (45K to 7K) and a 21.1% reduction in LLM calls (570K to 450K), as shown in Table 1. The MRT stress test in Table 2 further demonstrates the robustness of this reduced-order representation. Adding UCB to MemRL increases positive noise updates from 3.7 to 7.2 and the final noise ratio from 1.02% to 1.20%, whereas RoMeRL limits them to 2.4 and 0.15%, respectively, while achieving the highest success rate of 82.0%. Additional details are provided in Appendix B.

Cross-Model Memory Transfer. Table 3 shows the same pattern in all four model–task combinations: transferring the frozen memory state improves the score and reduces the average number of execution steps. The factorized state therefore carries procedural information that is useful across LLM backbones, both for solving more tasks and for reaching solutions more directly. Because each interaction step typically requires another LLM invocation, the lower step counts also reduce inference cost.

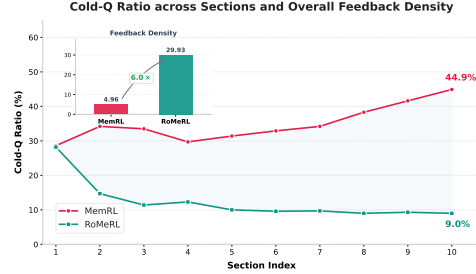


Figure 3: Feedback density and Cold-Q ratio on the OS task. The Cold-Q ratio is the fraction of current representatives with no direct utility update since admission, i.e., $n_i^{\text{post}} = 0$, while feedback density is their average number of post-admission updates.

Table 2: Controlled MRT stress test on the OS task. The first-round memory pool contains 10% noisy entries, and results are reported after ten training rounds. Positive Noise Updates denotes the average number of positive utility updates received by noisy entries over the ten rounds, whereas Final Noise Ratio denotes the percentage of noisy entries in the memory pool at the end of round 10.

Method	Round-10 SR (%) $\uparrow$	Positive Noise Updates $\downarrow$	Final Noise Ratio (%) $\downarrow$
MemRL	79.2	3.7	1.02
MemRL + UCB	78.4	7.2	1.20
RoMeRL	82.0	2.4	0.15

Table 3: Cross-model memory transfer on LifelongAgentBench OS and DB tasks. Results with and without frozen transferred memory are reported as **Validation Score / Average Steps** (higher / lower is better), and Δ denotes the absolute score gain over the base agent.

Inference Model	Base	Transfer	Gain (Δ)
<i>LIFELONGAGENTBENCH-OS</i>			
GPT-5.4-mini	67.0 / 3.23	81.6 / 2.22	+14.6 / -1.01
Gemini-3.5-flash	74.0 / 4.53	81.4 / 3.02	+7.4 / -1.51
<i>LIFELONGAGENTBENCH-DB</i>			
GPT-5.4-mini	93.0 / 2.15	96.8 / 2.00	+3.8 / -0.15
Gemini-3.5-flash	96.2 / 2.44	97.6 / 2.18	+1.4 / -0.26

6.2 ABLATION STUDY

To assess the factorized state, we ablate NCC, $z_{g,t}^{-,C}$, and PAC, $z_{g,t}^{+,A}$, on the OS task while retaining PCC and NAC as the basic positive and negative anchors. This isolates the complementary roles of consolidated negative evidence and adaptive positive transitions. The results are shown in Figure 5.

Negative Consolidated Coordinate. Removing NCC lowers both Last-Epoch Success Rate and CSR (Figure 4). NCC retains failure-derived experiences with high downstream raw-return utility, providing a compact source of potentially reusable negative information, allowing the agent to reuse informative negative evidence without storing every failed trajectory. In the full factorized state, NCC accounts for 27.14% of occupied coordinates on OS and 42.78% on DB. Its ablation and high occupancy together indicate that consolidated negative evidence supports both current performance and cumulative task coverage.

Positive Adaptive Coordinate. Removing PAC mainly reduces Last-Epoch Success Rate, with little change in CSR. PAC preserves the first successful trajectory observed after a failure, thereby retaining the recovery pattern. It occupies only 8.21% of the coordinates on OS and 5.05% on DB, but its removal still lowers current performance. PAC thus provides a sparse transition signal that helps the agent reproduce previously discovered solutions. The NCC and PAC results show complementary roles for consolidated negative evidence and adaptive positive evidence.

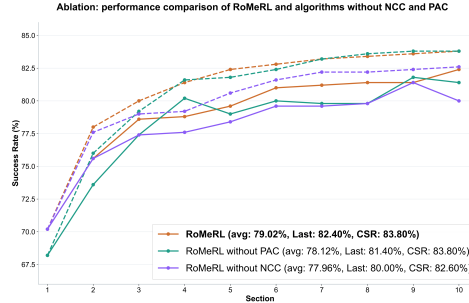


Figure 4: Coordinate ablation on the OS task. Solid and dashed curves denote SR and CSR.

7 CONCLUSION AND LIMITATIONS

We introduced RoMeRL, a reduced-order memory reinforcement learning framework for self-evolving LLM agents. Our analysis identifies the MRT, where outcome-driven Q updates reinforce

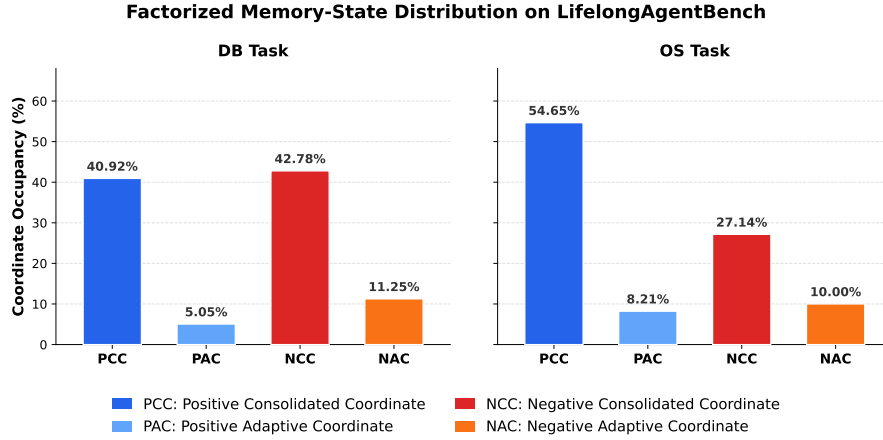


Figure 5: Occupancy distribution of the factorized memory state on the OS and DB tasks. The bars report the percentage of occupied active-memory coordinates assigned to the PCC, PAC, NCC, and NAC.

co-retrieved memories with little causal contribution. Exploration reduces estimation variance but not attribution bias, while full-pool learning requires growing feedback. RoMeRL instead uses compact per-task coordinates factorized by outcome polarity and memory dynamics, improving performance while reducing MRT exposure, memory size, and inference cost.

RoMeRL still relies on outcome-level rewards and therefore does not fully resolve causal credit assignment. Moreover, estimating the transition quantities γ and λ in Proposition 1 requires coordinate-level causal labels from paired counterfactual rollouts or equivalent attribution. Future work may combine such attribution with finer-grained rewards and evaluate open-ended, longer-horizon settings.

REFERENCES

- Zhicheng Cai, Xinyuan Guo, Yu Pei, Jiangtao Feng, Jinsong Su, Jiangjie Chen, Ya-Qin Zhang, Wei-Ying Ma, Mingxuan Wang, and Hao Zhou. Flex: Continuous agent evolution via forward learning from experience. *arXiv preprint arXiv:2511.06449*, 2025.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 17490–17502, 2026.
- Yao Fu, Dong-Ki Kim, Jaekyeom Kim, Sungryull Sohn, Lajanugen Logeswaran, Kyunghoon Bae, and Honglak Lee. Autoguide: Automated generation and selection of context-aware guidelines for large language model agents. *Advances in Neural Information Processing Systems*, 37:119919–119948, 2024.
- Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 32779–32798, 2025.
- Wei-Chieh Huang, Weizhi Zhang, Yueqing Liang, Yuanchen Bei, Yankai Chen, Tao Feng, Xinyu Pan, Zhen Tan, Yu Wang, Tianxin Wei, et al. Rethinking memory mechanisms of foundation agents in the second half: A survey. *arXiv preprint arXiv:2602.06052*, 2026.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pp. 6769–6781, 2020.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33: 9459–9474, 2020.
- Weitao Ma, Xiaocheng Feng, Lei Huang, Xiachong Feng, Zhanyu Ma, Jun Xu, Jiuchong Gao, Jinghua Hao, Renqing He, and Bing Qin. Fine-mem: Fine-grained feedback alignment for long-horizon memory management. *arXiv preprint arXiv:2601.08435*, 2026.
- Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T Le, Samira Daruki, Xiangru Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*, 2025.
- Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. Memgpt: towards llms as operating systems. 2023.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pp. 1–22, 2023.
- Rana Salama, Jason Cai, Michelle Yuan, Anna Currey, Monica Sunkara, Yi Zhang, and Yassine Benajiba. Meminsight: Autonomous memory augmentation for llm agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 33124–33140, 2025.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.

- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. *arXiv preprint arXiv:2010.03768*, 2020.
- Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*, 2023.
- Zhengwei Tao, Ting-En Lin, Xiancai Chen, Hangyu Li, Yuchuan Wu, Yongbin Li, Zhi Jin, Fei Huang, Dacheng Tao, and Jingren Zhou. A survey on self-evolution of large language models (2024). URL <https://arxiv.org/abs/2404.14387>, 2024.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandolekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024.
- F Wu, T Schuster, S Chen, et al. Rag: Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proc. 58th Annual Meeting of the Association for Computational Linguistics*, pp. 3030–3045, 2020.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38:17577–17604, 2026.
- Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z Pan, et al. Memory-rl: Enhancing large language model agents to manage and utilize memories via reinforcement learning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12805–12825, 2026.
- Yanyu Yao, Shangze Li, Zhi Zheng, Hui Zheng, Qi Liu, Tong Xu, and Enhong Chen. Atommem: Building simple and effective memory system for llm agents via atomic facts. *arXiv preprint arXiv:2606.19847*, 2026.
- Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. *arXiv preprint arXiv:2601.01885*, 2026.
- Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. Memevolve: Meta-evolution of agent memory systems. *arXiv preprint arXiv:2512.18746*, 2025a.
- Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Zhuo Li, Yujie Zheng, Weinan Zhang, Ying Wen, Zhiyu Li, et al. Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory. *arXiv preprint arXiv:2601.03192*, 2026.
- Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6):1–47, 2025b.
- Zeyu Zhang, Quanyu Dai, Rui Li, Xiaohe Bo, Xu Chen, and Zhenhua Dong. Learn to memorize: Optimizing llm-based agents with adaptive memory framework. *arXiv preprint arXiv:2508.16629*, 2025c.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19632–19642, 2024.
- Junhao Zheng, Xidi Cai, Qiuke Li, Duzhen Zhang, ZhongZhi Li, Yingying Zhang, Le Song, and Qianli Ma. Lifelongagentbench: Evaluating llm agents as lifelong learners. *arXiv preprint arXiv:2505.11942*, 2025.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pp. 19724–19731, 2024.

Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, et al. Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering. *arXiv preprint arXiv:2604.08224*, 2026.

Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, et al. Memento: Fine-tuning llm agents without fine-tuning llms. *arXiv preprint arXiv:2508.16153*, 2025.

Appendix

A PROOFS FOR RoMeRL

In this appendix, we provide detailed proofs for the theoretical results in Sections 3 and 4. We first prove the bias-variance decomposition of memory utility estimation, and then discuss its implication for outcome-driven memory updates.

A.1 PROOF OF THEOREM 1

Proof. Fix a memory m_i and a positive visit count n_i . Write the rewards observed on its retrieval events as $R_{i,1}, \dots, R_{i,n_i}$. Under the assumptions of Theorem 1, these variables are independent, with common mean μ_i and variance σ_i^2 , and

$$\hat{Q}_i = \frac{1}{n_i} \sum_{j=1}^{n_i} R_{i,j}.$$

Linearity of expectation and independence give

$$\begin{aligned} \mathbb{E}[\hat{Q}_i] &= \frac{1}{n_i} \sum_{j=1}^{n_i} \mathbb{E}[R_{i,j}] = \mu_i, \\ \text{Var}(\hat{Q}_i) &= \frac{1}{n_i^2} \sum_{j=1}^{n_i} \text{Var}(R_{i,j}) = \frac{\sigma_i^2}{n_i}. \end{aligned}$$

Thus, $\hat{Q}_i$ is centered at the observational utility rather than the interventional marginal utility. By definition, $a_i = \mu_i - v_i^1$ and $\mu_i - \theta_i = v_i^0 + a_i$. Adding and subtracting μ_i gives

$$\begin{aligned} \mathbb{E}[(\hat{Q}_i - v_i^1)^2] &= \mathbb{E}[(\hat{Q}_i - \mu_i)^2] + (\mu_i - v_i^1)^2 \\ &= \frac{\sigma_i^2}{n_i} + a_i^2, \\ \mathbb{E}[(\hat{Q}_i - \theta_i)^2] &= \mathbb{E}[(\hat{Q}_i - \mu_i)^2] + (\mu_i - \theta_i)^2 \\ &= \frac{\sigma_i^2}{n_i} + (v_i^0 + a_i)^2. \end{aligned}$$

The cross terms vanish because $\mathbb{E}[\hat{Q}_i - \mu_i] = 0$. □

Random visit counts. The theorem treats n_i as fixed. If the visit count is random, the same identity holds conditionally provided that, given n_i , the selected rewards retain the stated mean, variance, and independence properties. Under this non-informative sampling condition and on the event $n_i > 0$,

$$\mathbb{E}[(\hat{Q}_i - \theta_i)^2 \mid n_i] = (v_i^0 + a_i)^2 + \frac{\sigma_i^2}{n_i},$$

and averaging over n_i replaces the variance term by $\mathbb{E}[\sigma_i^2/n_i]$. This conditional step is not automatic under an adaptive stopping rule that depends on previously observed rewards; such a rule would require a separate martingale or optional-stopping analysis.

Why additional feedback does not remove the credit gap. For fixed v_i^0 and a_i , the variance term vanishes as n_i grows, whereas the total credit gap remains:

$$\lim_{n_i \rightarrow \infty} \mathbb{E}[(\hat{Q}_i - \theta_i)^2] = (v_i^0 + a_i)^2.$$

Moreover, $\hat{Q}_i \xrightarrow{p} \mu_i$. More feedback therefore makes the empirical estimate increasingly precise around the raw-return target μ_i ; it does not remove the task-level baseline v_i^0 , the observational attribution bias a_i , or their resulting gap to the marginal contribution θ_i .

Positive reinforcement under the memory-reward trap. Consider a trapped memory, for which $\theta_i \leq 0$ but $\mu_i - v_i^0 = \theta_i + a_i > 0$. Let $\mathcal{F}_t$ be the pre-reward history and define $\mu_{i,t} = \mathbb{E}[R_t \mid \mathcal{F}_t, m_i \in \mathcal{S}_t]$. Since $Q_{i,t}$ is $\mathcal{F}_t$ -measurable, the exponential update in Equation (4.1) satisfies

$$\mathbb{E}[Q_{i,t+1} - Q_{i,t} \mid \mathcal{F}_t, m_i \in \mathcal{S}_t] = \alpha(\mu_{i,t} - Q_{i,t}). \quad (22)$$

Under the stationary reference model $\mu_{i,t} = \mu_i$, the conditional drift is positive whenever $Q_{i,t} < \mu_i$. This raw-return drift includes the task baseline v_i^0 . The MRT is not identified by the positivity of μ_i alone, but by the sign reversal $\theta_i \leq 0 < \mu_i - v_i^0$. If the retrieval score is nondecreasing in Q_i , repeated raw-return updates can make such a memory more likely to be retrieved again.

Fixed-step EMA variance. Equation (22) is a statement about expected drift, not a vanishing-variance guarantee. To see the distinction, suppose the rewards at the n update events are i.i.d. with mean μ_i and variance σ_i^2 , and let $Q_{i,0}$ be deterministic. Unrolling the recursion gives

$$Q_{i,n} = (1 - \alpha)^n Q_{i,0} + \alpha \sum_{r=1}^n (1 - \alpha)^{n-r} R_{i,r}.$$

Consequently,

$$\begin{aligned} \mathbb{E}[Q_{i,n}] &= (1 - \alpha)^n Q_{i,0} + (1 - (1 - \alpha)^n) \mu_i \longrightarrow \mu_i, \\ \text{Var}(Q_{i,n}) &= \alpha^2 \sigma_i^2 \sum_{r=0}^{n-1} (1 - \alpha)^{2r} \\ &= \frac{\alpha \sigma_i^2}{2 - \alpha} (1 - (1 - \alpha)^{2n}) \longrightarrow \frac{\alpha \sigma_i^2}{2 - \alpha}. \end{aligned}$$

Thus, a fixed-step runtime EMA approaches μ_i in mean but generally retains a nonzero variance floor. The empirical-mean results in Theorems 1 and 3 concern a different estimator; the MRT argument for the runtime update relies only on its expected drift toward the observational target.

A.2 PROOF OF THEOREM 2

Proof. Write $N = N_t$ for the current full-pool dimension. For a fixed memory m_i , Hoeffding's inequality gives

$$\Pr(|\hat{Q}_i - \mu_i| > \epsilon) \leq 2 \exp(-2n_i \epsilon^2). \quad (23)$$

The estimates need not be independent across memories: applying the union bound directly yields

$$\Pr\left(\max_{i \in [N_t]} |\hat{Q}_i - \mu_i| > \epsilon\right) \leq \sum_{i=1}^{N_t} 2 \exp(-2n_i \epsilon^2).$$

It is therefore enough to choose a common per-memory count

$$n^* = \left\lceil \frac{1}{2\epsilon^2} \log \frac{2N_t}{\delta} \right\rceil \quad \text{and ensure } n_i \geq n^* \text{ for every } i. \quad (24)$$

Substitution into the union bound gives a failure probability no larger than δ , which proves the simultaneous guarantee in Equation (5).

It remains to translate the per-memory requirement into budgets. A balanced allocation that assigns n^* feedback signals to each of the N_t memories uses

$$F_T^* = N_t n^* \leq N_t \left(1 + \frac{1}{2\epsilon^2} \log \frac{2N_t}{\delta}\right).$$

Hence, in the nontrivial regime $0 < \epsilon \leq 1$ and $0 < \delta < 1$,

$$F_T^* = O\left(\frac{N_t}{\epsilon^2} \log \frac{N_t}{\delta}\right).$$

If at most k memory-level updates are packed into one trajectory, the same balanced allocation can be scheduled in $T^* = \lceil F_T^* / k \rceil$ trajectories. Thus

$$T^* = O\left(\frac{N_t}{k\epsilon^2} \log \frac{N_t}{\delta}\right),$$

up to the immaterial final partially filled trajectory. These are sufficient budgets, matching the statement of Theorem 2; no minimax lower bound is claimed. $\square$

A.3 PROOF OF THEOREM 3

Proof. Let D denote the number of utility coordinates being maintained, and let n_j be the number of feedback signals assigned to coordinate j . The realized number of coordinate-level updates is

$$F_T = \sum_{j=1}^D n_j \leq kT.$$

Irrespective of how these updates are distributed, the average count per coordinate is exactly

$$\bar{n}_D = \frac{1}{D} \sum_{j=1}^D n_j = \frac{F_T}{D}.$$

When the available update budget is used to constant order, $F_T \asymp kT$, this becomes $\bar{n}_D \asymp kT/D$, which is the approximation used in the theorem.

Now compare a trajectory-indexed state of dimension N_t with a factorized state of dimension four under the same realized feedback budget F_T . Their average feedback counts satisfy

$$\bar{n}_{\text{fact}} = \frac{F_T}{4} = \frac{N_t}{4} \frac{F_T}{N_t} = \frac{N_t}{4} \bar{n}_{\text{full}}. \quad (25)$$

This identity concerns feedback density and does not require identical coordinate-level visit counts.

The variance comparison does require the approximately balanced allocation stated in the theorem. More precisely, suppose there is a constant $c \in (0, 1]$, independent of D and T , such that $n_j \geq cF_T/D$ for every maintained coordinate. If the conditional reward variance of each coordinate is at most σ^2 , then the empirical-mean estimator from Theorem 1 obeys

$$\begin{aligned} \frac{1}{D} \sum_{j=1}^D \text{Var}(\hat{Q}_j) &\leq \frac{1}{D} \sum_{j=1}^D \frac{\sigma^2}{n_j} \\ &\leq \frac{\sigma^2 D}{cF_T} = O\left(\frac{\sigma^2 D}{kT}\right), \end{aligned}$$

where the final form again uses $F_T \asymp kT$. Setting $D = N_t$ gives $O(\sigma^2 N_t / (kT))$ for the trajectory-indexed state, whereas setting $D = 4$ gives $O(4\sigma^2 / (kT))$ for the factorized state.

If the update budget is not saturated, the same statements remain valid with F_T in place of kT . If feedback is highly unbalanced, the density identity in Equation (25) still holds, but average feedback alone no longer implies the stated variance bound. This is why the balanced-allocation condition is explicit in the theorem. $\square$

A.4 PROOF OF PROPOSITION 1

Proof. For the trajectory-indexed pool, define $X_i = \mathbb{1}[\theta_i \leq 0, \mu_i - v_i^0 > 0]$. By the definition of ρ_{full} , $\mathbb{E}[X_i] = \rho_{\text{full}}$. Therefore, without requiring independence among memories,

$$\mathbb{E}\left[\sum_{i=1}^{N_t} X_i\right] = \sum_{i=1}^{N_t} \mathbb{E}[X_i] = N_t \rho_{\text{full}}. \quad (26)$$

Now consider one coordinate of the active state. Let $p_{j,t}$ be the probability that coordinate j is erroneous after its t -th transition. An erroneous coordinate remains erroneous with probability at most $1 - \lambda$, while a clean coordinate becomes erroneous with probability at most γ . Hence

$$p_{j,t+1} \leq (1 - \lambda)p_{j,t} + \gamma(1 - p_{j,t}). \quad (27)$$

Under the stationary transition model in the theorem, write the stationary erroneous probability as p_j^* . Applying Equation (27) at stationarity and rearranging gives

$$(\gamma + \lambda)p_j^* \leq \gamma, \quad p_j^* \leq \frac{\gamma}{\gamma + \lambda}.$$

Table 4: Model and API Configurations.

Component	Configuration / Version	Notes
Backbone LLM	DS-V4-flash	Used for LifelongAgentBench
	GPT-5.4-mini	Used for ALFWorld
Embedding Model	Text-Embedding-3-Large	Used for Intent and Query encoding
Generation Params	Temperature = 0.0	General (Greedy decoding)
	Top-p = 1.0	Default

Let Y_j indicate that active coordinate j is erroneous in stationarity. Linearity of expectation again avoids any independence requirement:

$$\mathbb{E} \left[\sum_{j=1}^d Y_j \right] = \sum_{j=1}^d p_j^* \leq d \frac{\gamma}{\gamma + \lambda}. \quad (28)$$

Comparing Equations (26) and (28) proves that the active state has strictly smaller steady-state erroneous occupancy whenever

$$d \frac{\gamma}{\gamma + \lambda} < N_t \rho_{\text{full}}.$$

For a four-coordinate active state, setting $d = 4$ gives the corresponding conditional occupancy bound.

The replacement probability also controls persistence at the coordinate level. Conditional on a coordinate being erroneous, the probability that it remains erroneous for at least r further transitions is at most $(1 - \lambda)^r$. Its expected erroneous residence time is therefore at most $1/\lambda$. This residence-time bound is consistent with the stationary occupancy result in Proposition 1. $\square$

B IMPLEMENTATION DETAILS

We facilitate reproducibility by documenting the exact model versions, hyperparameter settings, and environmental configurations used in our experiments.

B.1 MODEL SPECIFICATIONS

We performed all LLM reasoning and generation tasks using the models in Table 4. To maximize reproducibility, we accessed them through the official APIs with a fixed temperature, ensuring deterministic outputs where feasible.

B.2 HYPERPARAMETER SETTINGS

Table 5 details the specific hyperparameters used for RoMeRL and the baselines. The similarity threshold δ is adaptive to the dataset density; specifically, we determine δ by calculating the pairwise cosine similarity distribution of task descriptions within each benchmark and selecting the threshold at the top 20% quantile. This ensures that only the most relevant historical experiences are considered for retrieval.

B.3 DATA PARTITIONING

To evaluate the effectiveness of RoMeRL, we categorize our experiments into Runtime Learning and Transfer Learning settings. Table 6 summarizes the dataset sizes and partitioning strategies used for each benchmark. For benchmarks utilizing random splits (OS, DB), we use a fixed random seed of 42 to ensure reproducibility.

Table 5: Hyperparameter Settings across Benchmarks.

Parameter	Description	Benchmark Setting		
		Lifelong Bench (OS)	Lifelong Bench (DB)	ALFWorld
<i>RoMeRL (Ours)</i>				
α	Learning Rate	0.3	0.3	0.3
ω_Q	Q-Weight Balance	0.5	0.5	0.5
δ	Similarity Threshold	0.50	0.37	0.62
k_1	Cosine Similarity Recall Size	10	10	5
k_2	Final Memory Selection Size	5	5	3
Q_{init}	Initial Q-value	0.5	0.5	0.0
<i>Baselines</i>				
k_{RAG}	Retrieval Top-k	5	5	3
$k_{SelfRAG}$	Retrieval Top-k	5	5	3
k_{MemP}	Retrieval Top-k	5	5	3

Table 6: Data split for Lifelong Agent benchmarks.

Benchmark	Runtime Learning	Transfer Learning	Split / Note
Lifelong Agent (OS)	500 tasks	500 tasks	7:3 Split (Seed 42)
Lifelong Agent (DB)	500 tasks	500 tasks	7:3 Split (Seed 42)

B.4 BENCHMARK DETAILS

We evaluate performance across two benchmarks covering operating-system and database interaction as well as embodied decision-making.

LifelongAgentBench (LAB) (Zheng et al., 2025): LifelongAgentBench is designed to evaluate lifelong learning and experience reuse in interactive terminal-based environments. It contains 1,396 total task instances across three environments: Database (DB), Operating System (OS), and Knowledge Graph (KG). Following prior work (Zhang et al., 2026), we focus on the DB and OS subsets. The DB subset (500 tasks) evaluates 22 SQL-related skills: basic SELECT, filtering (WHERE), grouping (GROUP BY), sorting (ORDER BY), aggregation (COUNT / SUM / AVG / MAX / MIN), nested subqueries, multi-table JOINS, set operations (UNION / INTERSECT), and data manipulation (INSERT / UPDATE / DELETE). Execution results are verified automatically via SQL engine output. The OS subset (500 tasks) evaluates 29 Bash-command skills: file and directory operations (ls, cp, mv, find), permission management (chmod, chown), user and group management (useradd, groupmod), text processing (grep, sed, awk, wc), compression (tar, gzip), process inspection (ps, top, kill), and system monitoring (df, du, uptime). Correctness is verified by checking final OS state.

ALFWorld (Shridhar et al., 2020): A text-based embodied household environment aligned with the ALFRED simulator. It translates household manipulation tasks into textual observations and actions while preserving long-horizon planning and partial observability. The benchmark includes six task types: *pick-and-place*, *examine-in-light*, *clean-and-place*, *heat-and-place*, *cool-and-place*, and *pick-two-and-place*. These require agents to locate objects, navigate between receptacles, manipulate object states, and place objects at target locations. The original split provides 3,553 training tasks, 140 validation-seen tasks, and 134 validation-unseen tasks.

B.5 CONTROLLED MRT STRESS TEST

To examine reward contamination in a controlled setting, we replace 10% of the first-round memory entries with noisy versions. These entries preserve their original titles but set the key action or reflection field to null. This construction retains the semantic cues used for retrieval while removing the actionable content, allowing the noisy entries to remain retrievable and potentially receive positive utility updates. Noise is injected only in the first round, after which all methods run for ten rounds under the same task and interaction budgets. For the MemRL+UCB variant, we set the UCB exploration coefficient to $c_{UCB} = 0.2$ and the maximum per-memory exploration bonus to $b_{max} = 0.3$. We report the round-10 success rate, the average number of positive utility updates

received by noisy entries, and the Final Noise Ratio, defined as the fraction of noisy entries in the memory pool at the end of round 10.

The Final Noise Ratio reflects both the persistence of the initially injected entries and the propagation of noise during subsequent interactions. In MemRL, we observe that exposure to a retrieved `null` entry leads to one additional memory containing the same `null` operation. The additional exploration induced by UCB amplifies this effect, explaining its higher final noise ratio relative to standard MemRL. In contrast, RoMeRL’s replacement mechanism replaces most noisy contents with higher-quality memories within the first three rounds, leaving only a small fraction of noisy entries at the end of training.

B.6 Q-VALUE STRATIFICATION AND FEEDBACK COVERAGE.

As shown in Figures 6, RoMeRL produces a substantially more informative final Q-value landscape than the MemRL baseline. The association between the learned Q-values and memory-generation provenance is markedly stronger under RoMeRL, with the point-biserial Pearson correlation increasing from $r = 0.493$ for the baseline to $r = 0.673$ for RoMeRL. Under RoMeRL, the proportion of success-derived memories increases from only 2.3% in the lowest Q-value bin (0.0–0.2) to 80.1% in the highest bin (0.9–1.0), demonstrating that the Critic learns a meaningful ranking signal over the complete memory pool. Although the baseline reaches a higher absolute success-derived proportion of 94.7% in its highest-Q bin, its memory pool already contains 77.3% success-derived memories overall, corresponding to only a $1.23\times$ enrichment. In contrast, RoMeRL raises the success-derived proportion from a pool-wide prevalence of 33.3% to 80.1% in the highest-Q bin, yielding a substantially stronger $2.40\times$ enrichment. More importantly, only 5.0% of RoMeRL memories remain at the initial $Q = 0.5$, compared with 47.8% for the baseline. This 42.8-percentage-point reduction indicates that RoMeRL exposes a much larger fraction of the final memory pool to value feedback, thereby avoiding the large uninformative default-Q plateau observed under the baseline.

B.7 UTILITY BEYOND BINARY SUCCESS REPLAY.

The composition of the highest-Q bin further reveals a qualitative difference between the two methods. While the baseline’s 0.9–1.0 bin is almost entirely composed of success-derived memories, retaining only 5.3% failure-derived memories, the corresponding RoMeRL bin retains approximately 19.9% failure-derived memories. Consequently, RoMeRL’s stronger correlation does not arise from simply copying the binary generation outcome into the Q-value: its high-Q region remains compositionally diverse while still being strongly enriched in success-derived memories. By itself, this compositional evidence shows that the learned Q-value is not a deterministic proxy for the success/failure label. Together with the concrete cases reported in Appendix D, where highly valued failure-derived memories contain reusable corrections, diagnostic information, or transferable procedural lessons, the result supports the interpretation that the Critic can recognize utility not reducible to binary episode outcomes. RoMeRL therefore provides a more expressive memory-retention signal than success-only replay: it combines substantially broader Q-update coverage with graded memory ranking, while preserving selected failure-derived experiences that may remain useful for future problem solving.

C COST AND EFFICIENCY ANALYSIS

C.1 TOKEN CONSUMPTION

We compare LLM-call cost across the full learning trajectory. Compared with MemRL, RoMeRL consistently requires fewer runtime calls, as shown in Figure 7. This improvement comes from the factorized per-task memory state. Instead of retrieving and updating memories from a continuously growing full pool, RoMeRL maintains a compact active support for each task. As a result, the agent is less likely to retrieve redundant, stale, or weakly relevant memories, and the prompt contains more targeted experience for the current task.

This role-guided mechanism reduces cost in two ways. First, RoMeRL avoids generating procedural memories for ineffective trajectories that cannot be promoted or replaced in the role-specific slots, which naturally reduces the number of LLM calls. Second, by providing more reliable success,

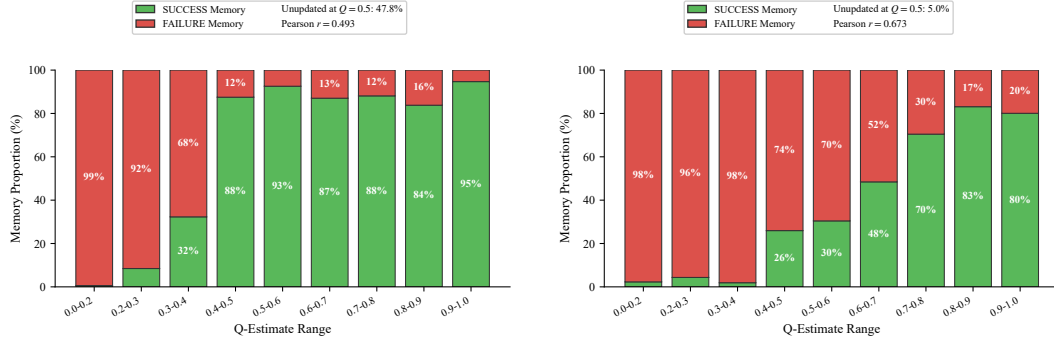


Figure 6: Final Q-value composition on OS tasks for the MemRL baseline (Left) and RoMeRL (Right). Each bar shows the proportions of success-derived and failure-derived memories within a Q-value interval; the legends report the fraction of memories remaining at the initial $Q = 0.5$ and the Pearson correlation between Q-values and memory-generation outcomes.

failure-diagnostic, and recovery information, RoMeRL helps the agent complete tasks in fewer interaction steps, which further reduces the total number of LLM calls. Therefore, the lower cost of RoMeRL is not merely a consequence of using fewer memories, but of replacing full-pool memory accumulation with a compact and role-structured active memory space.

On the LAB OS and DB tasks, RoMeRL achieves lower token consumption and fewer LLM calls than MemRL while maintaining stronger task performance. This indicates that role-guided memory replacement improves not only memory quality, but also the practical efficiency of non-parametric memory learning for autonomous agents.

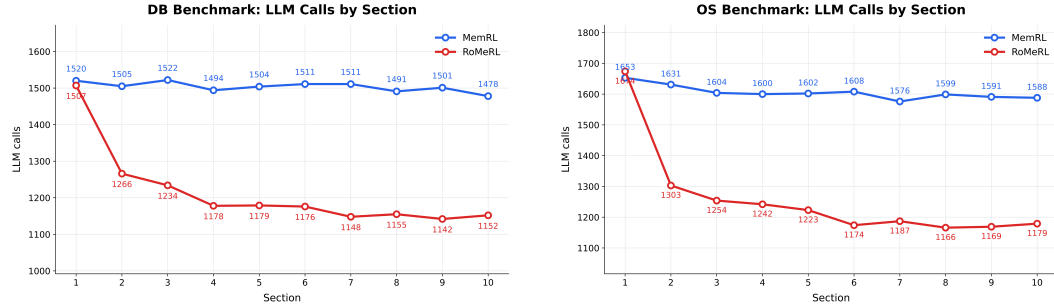


Figure 7: LLM calls per section on the DB and OS task. RoMeRL achieves lower and steadily decreasing call counts, while MemRL maintains a consistently high level.

D CASE STUDY: ROLE-BASED MEMORY ANALYSIS

This appendix presents qualitative case studies illustrating how RoMeRL’s four semantic coordinates are selected and utilized during the 10-epoch OS-interaction run. CS denotes *Case Study*. Each box below contains the origin task, the retrieved coordinate memory, a brief explanation of its contribution, and the target task where it was retrieved.

Case Study 1 (PCC — Stable reproduction of an efficient successful trajectory)

Coordinate: PCC.

Origin task.

```
> Create a directory `/target` containing symbolic links named `link1`
to `link5` pointing to `/source/file1` to `/source/file5`
respectively. Ensure the directory is owned by group `testgroup`
with permissions 775.
```

Retrieved success memory.

```
- #1: [MEMORY TYPE] PROCEDURE
Task: Create a directory '/target' containing symbolic links named '
link1'
to 'link5' pointing to '/source/file1' to '/source/file5' respectively.
Ensure the directory is owned by group 'testgroup' with permissions 775.

SCRIPT:
### High-Level Script

1. Ensure the target group exists.
2. Create the target directory with `mkdir -p`.
3. Set group ownership and permissions with `chgrp` and `chmod`.
4. Create the five symbolic links using absolute source paths.
5. Verify the result with `ls -la /target`.

TRAJECTORY:
assistant: Ensure the group exists, create /target, set its group and
mode,
create link1 through link5, and verify the directory.
assistant: Act: bash
assistant: Act: finish
```

Explanation. This is a standard use of the **PCC**: the target task and the memory task are identical, and the retained successful trajectory has been compressed into a concise, idempotent, and verifiable operation template. It has a semantic similarity of 1 and maintains a high Q-value after subsequent feedback. The task is again completed in two steps in the tenth evaluation section, indicating stable reproduction of a verified solution rather than renewed exploration.

Target task (Same task; retrieval rank 1; run outcome: correct).

```
Create a directory '/target' containing symbolic links named 'link1' to
'link5' pointing to '/source/file1' to '/source/file5' respectively.
Ensure
the directory is owned by group 'testgroup' with permissions 775.
```

Case Study 2 (NAC — Recent failures provide warnings but remain vulnerable to false credit)

Coordinate: NAC.

Origin task.

```
Search for all lines containing 'ERROR' in .log files within /var/log/
app,
compile a summary report in /var/log/app/summary.txt with error counts
per
file and list affected files, then move processed logs to
/var/log/app/archive.
```

Retrieved reflection memory.

```
- #1: [MEMORY TYPE] FAILURE_REFLECTION
[TASK]
Search for all lines containing 'ERROR' in .log files within /var/log/
app,
compile a summary report in /var/log/app/summary.txt with error counts
per
file and list affected files, then move processed logs to
/var/log/app/archive.

[REFLECTION]
- ROOT CAUSE: The assistant assumed that /var/log/app existed and was
writable, and used a non-recursive glob over *.log. The task may also
contain case-sensitive matching and output-format requirements that
were
not explicitly validated.
- PATTERN TO AVOID: Do not rely on a simple glob or on a successful-
looking
shell transcript. Check the directory, collect the intended files
explicitly, and verify the exact report format and archive state.
- CORRECT APPROACH: Validate the input directory and permissions, use a
controlled file list, count ERROR lines per file, write the required
summary format exactly, move the processed logs, remove temporary
files,
and verify every evaluator-visible artifact.
```

Explanation. This case study demonstrates both the value and risk of the **NAC**. It immediately exposes the vulnerabilities of the most recent failed scenario and provides a counterexample for a similar task. However, its content has not been validated, and a bundle-level Q update shared across co-retrieved coordinates may assign a successful target-task outcome to this failure memory.

Target task (retrieval rank 4; run outcome: correct).

```
Process all error logs in /var/log/app: extract 'ERROR' lines into
errors\_summary.log, write the count to error\_count.txt, create a
backup,
set permissions to 644, and move original logs into a backup directory.
```

Case Study 3 (NCC — Preserving diagnostically valuable failure experience)

Coordinate: NCC.

Origin task.

Create a user 'devuser', create groups 'dev', 'qa', and 'ops', add the user to all three groups, set the login shell to /bin/zsh, and ensure the directory '/project' exists with group ownership 'dev' and permissions 770.

Retrieved reflection memory.

```
- #1: [MEMORY TYPE] FAILURE_REFLECTION
[TASK]
Create a user 'devuser', create groups 'dev', 'qa', and 'ops', add the
user to all three groups, set their login shell to /bin/zsh, and ensure
the
directory '/project' exists with group ownership 'dev' and permissions
770.

[REFLECTION]
- ROOT CAUSE: The previous solution did not explicitly ensure that /bin/
zsh
was installed and listed in /etc/shells before calling chsh. It also
used
fallback logic that could silently mask user-creation or package-
install
failures.
- PATTERN TO AVOID: Do not assume that installing a shell makes it a
valid
login shell, and do not suppress errors from privileged setup commands
.
- CORRECT APPROACH: Verify or install zsh, ensure /bin/zsh is present in
/etc/shells, create or update the user and all groups idempotently,
set the
shell, configure /project as dev:dev with mode 770, and verify every
state.
```

Explanation. This case demonstrates the transferability of the NCC. Origin task 289 discusses 'devuser', 'qa', '/project', and '/bin/zsh', while target task 103 discusses 'appuser', 'admins', '/var/app/app.log', and '/bin/bash'. Although the task entities differ, they share the operational structure of user/group creation, shell modification, permission setting, and explicit verification.

Target task (retrieval rank 1; run outcome: correct).

Create groups 'dev', 'ops', and 'admins'; add user 'appuser' to all three groups, set their login shell to /bin/bash, enforce password expiry in 30 days, create /var/app/app.log, make the log file group-readable, and make the home directory group-accessible.

Case Study 4 (PAC — Retaining the first successful recovery after failure)

Coordinate: PAC.

Origin task.

```
Create a group 'devteam', add users 'alice' and 'bob' to it, set 'alice'
as
the group administrator, and configure a shared directory '/'
devteam_shared'
accessible only by the group.
```

Retrieved success memory.

```
- #1: [MEMORY TYPE] PROCEDURE [COORDINATES: PCC + PAC]
Task: Create a group 'devteam', add users 'alice' and 'bob' to it, set '
alice' as the group administrator, and configure a shared directory
'/devteam_shared' accessible only by the group.

SCRIPT:
### High-Level Script for Setting Up a Group-Administered Shared
Directory

1. Ensure the group and both target users exist.
2. Add alice and bob to devteam and set alice as administrator with `
gpasswd -A alice devteam`.
3. Create /devteam_shared, set its group to devteam, and use mode 770.
4. Verify membership, administrator assignment, group ownership, and
mode.

TRAJECTORY:
assistant: Create the group and users, configure membership and the
group administrator, create the 770 shared directory, and verify all
conditions.
assistant: Act: bash
assistant: Act: finish
```

Explanation. The stored memory is a successful procedure. It occupies the **PAC** because this trajectory was the first success after task 114's earlier failure; the same trajectory may also occupy the PCC when it is the most efficient successful representative. It is not a separate failure-reflection record.

Target task (Same task; retrieval rank 1; run outcome: correct).

```
Create a group 'devteam', add users 'alice' and 'bob' to it, set 'alice'
as
the group administrator, and configure a shared directory '/'
devteam_shared'
accessible only by the group.
```


E PROMPT DETAILS

We provide the exact prompt strings and message templates used by our RoMeRL implementation across all benchmarks. For clarity, we separate the prompts that summarize experiences into memories from those used at task time for generation and inference.

E.1 EXPERIENCE SUMMARIZATION PROMPTS

ALFWorld: Experience Summarization Prompts

Trajectory serialization (stored as the episode trajectory).

The full ALFWorld dialogue history is stored as the trajectory:
`List[{"role": ..., "content": ...}, ...]`
 When interpolated into the summarization prompts, it is treated as a string representation.

High-level script generation prompt.

Analyze the following detailed task trajectory and create a concise, high-level script that captures the essential steps and decision points.

The script should be:

1. Generic enough to apply to similar tasks
2. Specific enough to provide useful guidance
3. 3-5 high-level steps maximum
4. Focus on the strategy and key decisions, not detailed actions

Trajectory:

`{trajectory}`

High-level script:

Failure reflection prompt.

Task: `{task_description}`

Failed trajectory:
`{failed_trajectory}`

This task failed. Analyze what went wrong and suggest improvements for future similar tasks.

Focus on:

1. Incorrect assumptions
2. Steps to improve
3. What to avoid next time

Provide a brief reflection:

Stored memory content templates.

```

# Successful memory
Task: {task_description}

SCRIPT:
{script}

TRAJECTORY:
{trajectory}

# Failure memory
TASK REFLECTION:
Task: {task_description}

What went wrong:
{reflection}

Failed approach:
{failed_trajectory}

```

LLB (LifelongAgentBench): Experience Summarization Prompts

Trajectory serialization (stored as the episode trajectory).

```

{role_1}: {content_1}
{role_2}: {content_2}
...

```

High-level script generation prompt.

Analyze the following detailed task trajectory and create a concise, high-level script that captures the essential steps and decision points.

The script should be:

1. Generic enough to apply to similar tasks
2. Specific enough to provide useful guidance
3. 3-5 high-level steps maximum
4. Focus on the strategy and key decisions, not detailed actions

Trajectory:
{trajectory}

High-level script:

Failure reflection prompt.

Task: {task_description}

Failed trajectory:
{failed_trajectory}

This task failed. Analyze what went wrong and suggest improvements for future similar tasks.

Focus on:

1. Incorrect assumptions
2. Steps to improve
3. What to avoid next time

Provide a brief reflection:

Stored memory content templates.

Successful memory
Task: {task_description}

SCRIPT:
{script}

TRAJECTORY:
{trajectory}

Failure memory
TASK REFLECTION:
Task: {task_description}

What went wrong:
{reflection}

Failed approach:
{failed_trajectory}

E.2 GENERATION AND INFERENCE PROMPTS

ALFWorld: Generation and Inference Prompts

Base system prompt (ReAct format + action space).

Interact with a household to solve a task. Imagine you are an intelligent agent in a household environment and your target is to perform actions to complete the task goal. At the beginning of your interactions, you will be given the detailed description of the current environment and your goal to accomplish.

For each of your turn, you will be given the observation of the last turn. You should first think about the current condition and plan for your future actions, and then output your action in this turn. Your output must strictly follow this format: "Thought: your thoughts .\nAction: your next action".

The available actions are:

1. go to {recep}
2. take {obj} from {recep}
3. move {obj} to {recep}
4. open {recep}
5. close {recep}
6. use {obj}
7. clean {obj} with {recep}
8. heat {obj} with {recep}
9. cool {obj} with {recep}

where {obj} and {recep} correspond to objects and receptacles.

After your each turn, the environment will give you immediate feedback based on which you plan your next few steps. if the environment output "Nothing happened", that means the previous action is invalid and you should try more options.

Your response should use the following format:

Thought: <your thoughts>
Action: <your next action>

Retrieved memory injection (system message).

In addition to the example, you have the following memories from your own past experiences. Use them to help you if they are relevant:

--- SUCCESSFUL MEMORIES (Examples to follow) ---
{formatted_success_memories_joined}

--- FAILED MEMORIES (Examples to avoid or learn from) ---
{formatted_failed_memories_joined}

Current task prompt (user message).

Now, it's your turn to solve a new task.
{task_description}

Per-step observation prompt (user message).

Observation: {observation}

Message ordering (high level).

- 1) system: base ALFWorld system prompt
- 2) user/assistant: selected few-shot example dialogue (sequence of messages)
- 3) system: optional retrieved memory context
- 4) user: new task prompt
- 5) loop: append user Observation: ..., model replies with Thought/Action

LLB (LifelongAgentBench): Generation and Inference Prompts

Base system prompt.

You are an execution-focused AI agent solving database and operating-system tasks.

You may receive a [Retrieved Memory Context] block with past experiences from similar problems.

These are ****references for learning****, not guaranteed solutions:

- [MEMORY TYPE] SUCCESS_PROCEDURE: A successful approach from a similar task learn the pattern.
- [MEMORY TYPE] FAILURE_REFLECTION: A failed attempt with lessons avoid similar mistakes.

Use the memories as inspiration, but always analyze your current task independently and adapt your approach based on its specific requirements.

Strict output constraint (DB tasks).

STRICT OUTPUT FORMAT (LLB:DB, do not violate):

- 1) After your reasoning, include exactly ONE action line:
 - Action: Operation
 - Action: Answer
- 2) If Action: Operation, put exactly ONE SQL statement in the FIRST fenced code block using ````sql`, on a single line. Do not add any extra text after that block.
- 3) If Action: Answer, include 'Final Answer: ...' on the next line and do not add extra text after that.

Strict output constraint (OS tasks).

STRICT OUTPUT FORMAT (LLB:OS, do not violate):

- 1) After your reasoning, include exactly ONE action line:
 - Act: bash
 - Act: finish
- 2) If Act: bash, the next lines MUST be a ````bash` fenced code block with your Bash commands. Do not include any other code blocks.
- 3) If Act: finish, it must be the last line (no code blocks, no extra text).
- 4) Do NOT use 'Action:' in OS tasks (use 'Act:' only).

Retrieved memory injection block.

[Retrieved Memory Context]

=== SUCCESSFUL EXPERIENCES (Learn from these) ===

[SUCCESS 1] [TYPE: {mem_type}]
{content}

=== FAILED EXPERIENCES (Avoid these mistakes) ===

[FAILURE 1] [TYPE: {mem_type}]
{content}

Prompt assembly ordering (system prompt).

- 1) base system prompt
- 2) optional [Retrieved Memory Context] ...
- 3) strict output format block appended at the end (task-aligned)